

Reinforcement Learning Project
Master's in data science and advanced Analytics

NOVA Information Management School
Universidade Nova de Lisboa

RL Agents on ICU Sepsis

Group B

Miguel Matos, 20221925

Ana Margarida Macedo, 20250405

Mehmet Karaca, 20250344

Veronica Mendes, 20221945

Luis Mendes, 20221949

Spring Semester 2025-2026

Table of Contents

1. Introduction	3
2. Methodology	4
2.1 Configuration A: Discrete State Space	5
2.1.1 Dynamic programming (DP)	6
2.1.2 Q- Learning	6
2.1.3 SARSA	7
2.2 Configuration B: Continuous Observation Space	8
2.2.1 Deep Q-Network (DQN)	8
2.2.2 Proximal Policy Optimisation (PPO)	9
2.2.3 Hyperparameter Search	9
3. Evaluation	9
3.1 Configuration A: Discrete State Space	10
3.1.1 Dynamic Programming	10
3.1.2 Q- Learning	10
3.1.3 SARSA	10
Interpretability	15
4. Conclusion	17
5. References	20
6. Annex	21

1. Introduction

(Explain the clinical problem, why RL is appropriate, and your overall approach.)

Sepsis is one of the leading causes of mortality in intensive care units (ICUs) worldwide, accounting for millions of fatalities each year. It is a complex, life-threatening condition triggered by the body's overwhelming response to an infection. Managing a septic patient requires continuous, high-stakes medical decisions centered around balancing two primary interventions: administering vasopressor medications to stabilize blood pressure and providing intravenous (IV) fluids to maintain systemic circulation. This clinical management is a delicate balancing act; administering too little intervention allows the patient to rapidly deteriorate, while administering too much can cause severe secondary harm and treatment-induced toxicity. Historically, these nuanced, hour-by-hour decisions have been made by ICU clinicians relying heavily on subjective personal experience and generic clinical guidelines.

The inherent limitations of static clinical guidelines become apparent when confronted with the vast complexity of sepsis progression, which features hundreds of evolving patient states, dozens of potential intervention actions, highly delayed physiological outcomes, and extreme individual patient variability. This multi-dimensional, sequential decision-making landscape under uncertainty makes sepsis treatment a natural candidate for Reinforcement Learning (RL). Unlike traditional statistical models, a well-trained RL agent can learn dynamically optimized treatment policies directly from historical patient data. By modeling the ICU environment, the agent can map diverse clinical trajectories and uncover complex, personalized therapeutic strategies designed to maximize terminal survival rates while simultaneously minimizing unnecessary, high-intensity interventions.

In this project, we address this sequential optimization challenge by developing and evaluating reinforcement learning agents on the [ICU-Sepsis-v2](#) benchmark, an environment built directly from the real-world MIMIC-III clinical database. Each training and evaluation episode simulates an individual ICU patient trajectory, where the agent observes the patient's clinical variables at each step and selects an intervention combining vasopressor and IV fluid dosage levels. The episode terminates when the patient either survives (yielding a terminal reward of +1.0) or dies (yielding a reward of 0); the primary objective of our agents is to maximize this survival rate while simultaneously minimizing unnecessary treatment intensity. To evaluate how reinforcement learning paradigms scale, our approach is structured around two distinct configurations of this underlying environment. In Configuration A, we exploit a finite state space of 716 discrete entries where the full MDP transition and reward matrices are accessible, allowing us to implement a combination of model-based planning alongside model-free, on-policy and off-policy tabular methods. In Configuration B, we transition to a highly realistic, 47-dimensional continuous observation space further complicated by operational clinical wrappers, necessitating deep function approximation through value-based off-policy and policy-gradient on-policy architectures. Ultimately, our comparative analysis tracks learning curves, convergence speeds, and exploration-exploitation dynamics across both environmental setups.

(Fill with our results at the end) Empirically, our results demonstrate that in the discrete domain of Configuration A, model-based Dynamic Programming established the absolute performance ceiling, while tabular Q-Learning achieved stable convergence within 10,000 episodes, marginally outperforming SARSA's more conservative policy. In the continuous and noisy setting of Configuration B, Proximal Policy Optimisation (PPO) proved significantly more robust than Deep Q-Networks (DQN); PPO exhibited steady, monotonic learning curves and achieved a final patient survival rate of 84%, whereas DQN suffered from severe value overestimation and optimization instability due to the missing data and observation noise. Clinically, the optimal learned policies across both configurations successfully adapted to the treatment intensity penalty. Rather than sustaining aggressive, high-dosage interventions, the best-performing agents discovered a parsimonious strategy: they initiated low-to-medium IV fluid volumes at the earliest signs of physiological distress, while strictly reserving high-tier vasopressor administration for acute, critical drops in blood pressure. This behavior effectively balanced immediate patient stabilization with long-term survival, mirroring safe and optimized intensive care protocols.

1.2. Environment & Problem Setup (~0.5 page)

****ICU-Sepsis-v2**** is a discrete-time MDP benchmark constructed from the MIMIC-III clinical database (Komorowski et al., 2018). Each episode simulates one ICU patient trajectory: at every step the agent observes the patient's current state, selects a treatment action, receives a reward signal, and transitions to the next state until the episode terminates in either survival or death. The action space is shared across both configurations: ``Discrete(25)``, representing every combination of five vasopressor dose levels and five IV fluid dose levels (none / low / medium / high / very high). Action 0 prescribes no treatment; action 12 prescribes medium doses of both interventions; action 24 prescribes maximum doses of both. To discourage unnecessary over-treatment, a per-step intensity penalty ``lam = 0.02`` is subtracted from the reward at each timestep — so the effective reward is $+1 - (0.02 \times \text{intensity})$ at survival, and $-(0.02 \times \text{accumulated intensity})$ at death. This means mean episode return is always slightly below survival rate, with the gap reflecting total treatment cost.

The two configurations differ only in how the patient state is represented. ****Configuration A**** encodes the patient's condition as a single discrete integer in the range `[0, 715]`, derived by discretising the raw physiological measurements into 716 clinically defined categories. Because the state space is finite and the full MDP — transition probability tensor $P(s'|s,a)$ and reward tensor $R(s,a,s')$ — is accessible from the environment internals, model-based planning methods such as Dynamic Programming can be applied directly. A Q-table for this configuration requires only 17,900 entries (716 states $\times$ 25 actions), making tabular model-free methods entirely tractable. ****Configuration B**** replaces the discrete index with the raw 47-dimensional physiological feature vector used in the original AI Clinician study: SOFA score, heart rate, mean arterial pressure, lactate, creatinine, and 42 additional clinical variables. The observation space is now continuous — the agent never encounters the exact same state twice — rendering a finite Q-table computationally infeasible. Three clinical failure wrappers additionally stress-test the agents: ``EpisodicNoisyObsEnv`` applies Gaussian noise to all observations for randomly selected episodes (15% of episodes); ``EpisodicMissingObsEnv`` zeroes out four lab features for

the duration of an episode (15% of episodes); and `AcuteEventEnv` introduces a 1% per-step probability of sudden irreversible patient death independent of any treatment decision.

****Random baseline.**** Before training any agent, a uniformly random treatment policy is evaluated over 1,000 episodes (fixed seed 42) to establish the performance floor. In Configuration A this policy achieves ****74.3% survival****; in Configuration B under the full clinical wrapper stack it achieves ****68.3% survival****. The lower Config B baseline reflects the irreducible mortality imposed by the `AcuteEventEnv` wrapper: with ~10 steps per episode, roughly 9% of patients die regardless of the chosen treatment. All trained agents are evaluated against their respective random baseline, and the delta above random is used as the primary cross-configuration comparison metric throughout Section 3.

2. Methodology

(Explain Description of each algorithm (policy or off policy, model based or free, function approximation or not, bootstrap, ect), why you chose it (pros, con), and key hyperparameters.)

To systematically address the high-stakes, sequential decision-making challenge of clinical sepsis management, our methodology evaluates agent performance across two distinct environmental setups designed to simulate patient trajectories. In both configurations, the underlying medical objective remains the same: discovering a therapeutic policy that stabilizes a patient's vital signs by carefully regulating the balance between vasopressor medication and intravenous (IV) fluid administration. To achieve this, both setups share a unified action space modeled as a **Discrete(25)** distribution, representing every combination of five escalating dosage tiers (none, low, medium, high, and very high) for each of the two treatments. For example, selecting action 0 commands a completely passive approach with no vasopressor and no IV fluid administered, action 12 applies a moderate intervention consisting of medium vasopressor and medium fluid levels, while action 24 represents an aggressive, maximum-intensity treatment of very high doses for both interventions. These clinical actions are evaluated first under Configuration A, which abstracts patient states into a finite space of 716 discrete indices, and subsequently under Configuration B, which elevates the complexity to a 47-dimensional continuous observation space mapping raw physiological variables and realistic clinical data corruptions.

Our analytical pipeline begins with the implementation of a completely random action selection policy, which establishes a baseline benchmark to quantify the floor performance of a non-learning agent operating blindly within the ICU environment. Moving beyond this random baseline, we deploy specialized Reinforcement Learning (RL) frameworks tailored to the structural representation of each configuration, enabling the models to learn the optimal course of medical action directly through environmental interactions to maximize patient survival. For every selected algorithm, a comprehensive optimization process is conducted to isolate the exact hyperparameter configurations: spanning learning rates, exploration-exploitation mechanics, and underlying updating or network architectures; that yield stable therapeutic

policies. We then perform an in-depth convergence analysis and learning curve evaluation across all models, capturing and visualizing metrics such as cumulative return per episode, survival success rates, and training velocity. This evaluation allows us to mathematically track how effectively both tabular and deep function approximation paradigms transition from initial unguided exploration to safe policy exploitation, ensuring that our final algorithmic comparisons are both statistically rigorous and clinically interpretable.

2.1 Configuration A: Discrete State Space

Configuration A abstracts the patient's complex physiological condition into a single discrete integer ranging from 0 to 715. Because the state space is strictly finite and paired with 25 possible actions, a comprehensive Q-table requires only 17,900 entries. This manageable dimensionality makes traditional tabular Reinforcement Learning methods entirely feasible, as exact state-action values can be stored and updated directly without the risk of generalization errors. Furthermore, because the full Markov Decision Process (MDP) model—including the exact transition probabilities and reward matrices—is accessible, model-based planning approaches can be deployed alongside model-free alternatives.

We selected Dynamic Programming (DP), Q-Learning, and SARSA as our three core algorithms. This selection is highly deliberate: it spans fundamentally different RL paradigms within the tabular domain. By incorporating these three methods, we can directly contrast model-based planning (DP) against model-free learning, while simultaneously evaluating the empirical trade-offs between off-policy updates (Q-Learning) and on-policy updates (SARSA) in a safety-critical clinical environment.

2.1.1 Dynamic programming (DP)

Dynamic Programming (DP) is a model-based method that computes the optimal policy directly from the known transition matrix $P(s'|s,a)$ and reward matrix $R(s,a,s')$, without requiring environment interaction during training. This makes it well-suited to ICU-Sepsis, where full MDP dynamics are available and unnecessary exploration raises patient safety concerns.

Two algorithms were implemented: Policy Iteration, which alternates between full policy evaluation (Bellman expectation equation) and greedy policy improvement until the policy stabilises and Value Iteration, which combines both steps into a single Bellman optimality update per iteration. Both used a convergence threshold of $1e-6$ and were tested across $\gamma \in \{0.90, 0.95, 0.99, 1.0\}$, with $\gamma = 1.0$ adopted as the final setting following the ICU-Sepsis paper convention.

Bellman expectation equation (Policy Iteration): $V(s) = \sum_{s'} P(s'|s,a) \cdot [R(s,a) + \gamma \cdot V(s')]$

Bellman optimality equation (Value Iteration): $V(s) = \max_a \sum_{s'} P(s'|s,a) \cdot [R(s,a) + \gamma \cdot V(s')]$

Bellman expectation equation (Policy Iteration): $V(s) = \sum_{s'} P(s'|s,a) \cdot [R(s,a) + \gamma \cdot V(s')]$

Bellman optimality equation (Value Iteration):

$$V(s) = \max_a \sum_{s'} \{s'\} P(s'|s, a) \cdot [R(s, a) + \gamma \cdot V(s')]$$

2.1.2 Q- Learning

Q-Learning is a model-free, off-policy temporal difference algorithm that learns the optimal action-value function $Q(s, a)$ directly from environmental interactions, without requiring access to transition probabilities or reward matrices. At each step, the agent observes a state, selects an action, receives a reward, and updates its Q-table according to the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max Q(s', a') - Q(s, a)]$$

The off-policy nature of Q-Learning means the update target always bootstraps from the greedy maximum over next-state values, regardless of which action the agent actually takes next. This allows the algorithm to learn the optimal policy while simultaneously following an exploratory behaviour policy, making it particularly well suited to the sepsis management task where safe exploration and optimal exploitation must be carefully balanced.

Exploration is governed by an ϵ -greedy strategy. Critically, our implementation enforces a dedicated exploration phase spanning the first 2,000 episodes, during which ϵ is held fixed at 1.0 to encourage broad coverage of the 716-state space before any exploitation begins. After this phase, ϵ decays linearly over a configurable number of steps toward a minimum value ϵ_{end} , progressively shifting the agent toward greedy action selection as the Q-table matures.

The key hyperparameters governing Q-Learning behaviour are the learning rate α , which controls how aggressively new information overrides existing estimates; the discount factor γ , which balances the weight of immediate versus future rewards; the decay schedule ϵ_{decay} , which determines the speed of the exploration-to-exploitation transition; and ϵ_{end} , which sets the floor on residual exploration maintained throughout training. To identify the optimal combination of these parameters, a systematic hyperparameter search was conducted using Optuna with a Tree-structured Parzen Estimator (TPE) sampler, evaluating each candidate configuration over 5,000 training episodes and selecting the configuration that maximised patient survival rate across validation episodes.

2.1.3 SARSA

SARSA is an on-policy Temporal Difference (TD) learning algorithm used to estimate the action-value function $Q(s, a)$ for the discrete state-space configuration on the ICU-Sepsis. Unlike off-policy algorithms like Q-learning, which optimize purely for the theoretical optimal path assuming perfect exploitation, SARSA updates its action-value estimates using the actual action

selected by the current behavioral policy (ϵ -greedy). The updates follow the classic TD(0) update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Where:

- s, a are the current state and chosen treatment action.
- r is the clinical reward feedback from the environment.
- s', a' are the subsequent state and the actual next action selected by the agent's ϵ -greedy exploratory schedule.

Because SARSA updates its memory based on the actual next action it chooses, it is naturally safer during training than algorithms like Q-Learning. In a high-stakes ICU environment where dangerous medical decisions (such as mismatched drug dosages) can severely harm a patient, SARSA avoids risky behaviors by accounting for its own random mistakes while exploring. To find the best setup for this agent, an automated Optuna grid search maximized the patient's survival rewards by evaluating the parameters listed in the Annex (Table 1). This search tuned the learning rate (α) and its decay schedule to control how fast the agent learns from new patient data, as well as the discount factor (γ) to balance immediate stabilization with long-term recovery. It also optimized the exploration schedule (ϵ -greedy) to safely guide the agent from early trial-and-error treatment to final, confident medical decisions. Crucially, the search tested different initial memory settings (Q-table initialization (Q_{init})) to see if changing the agent's starting expectations would improve how it explores different treatment paths. Looking at these initial settings broadly allowed the optimization process to analyze whether pre-existing assumptions or complete neutrality helps an algorithm uncover safer medical strategies.

2.2 Configuration B: Continuous Observation Space

Configuration B exposes the agent to the raw 47-dimensional physiological feature vector from the MIMIC-III database: SOFA score, heart rate, lactate, blood pressure, creatinine, and 42 additional clinical variables. Because the observation space is now continuous, the agent never encounters the exact same state twice. Tabular methods such as Q-Learning and SARSA are no longer applicable — a table indexed by state would require infinitely many entries. Function approximation via neural networks becomes necessary.

To closely replicate real-world intensive care conditions, the baseline environment is modified using three distinct operational wrappers that inject realistic observational vulnerabilities and clinical entropy. First, the **Episodic Observation Noise** wrapper (`EpisodicNoisyObsEnv`) applies random Gaussian noise to all physiological readings across randomly selected episodes. This introduces simulated equipment calibration shifts or sensor placement errors, forcing the agent to remain stable under degraded data conditions. Second, the **Episodic Missing Observations** wrapper (`EpisodicMissingObsEnv`) completely zeroes out a fixed, random subset of clinical variables for the duration of an episode. This simulates common ICU challenges like lab analyzer downtime or missing admission charts, requiring the agent to make safe therapeutic decisions under incomplete information. Finally, the **Acute Clinical Events**

wrapper (**AcuteEventEnv**) introduces a low-probability step function that terminates an episode with a patient death outcome entirely independent of the agent's actions. This injects irreducible clinical randomness—such as a sudden cardiac arrest or massive embolism—which prevents the model from falsely associating unpredictable biological failure with its selected treatment policy.

We selected **Deep Q-Network (DQN)** and **Proximal Policy Optimisation (PPO)** as our two algorithms. This selection is mathematically deliberate, as it directly contrasts a value-based, off-policy approach (DQN) against a policy-gradient, on-policy framework (PPO). Both are implemented using Stable-Baselines3 with custom MLP policies and trained with $\gamma = 1.0$, matching the environment convention of undiscounted episodic returns.

2.2.1 Deep Q-Network (DQN)

DQN is an off-policy, value-based method. It approximates the action-value function $Q(s, a)$ with a neural network and trains on mini-batches drawn from an experience replay buffer. Two mechanisms address the instability that arises when neural networks are used as function approximators: (i) the replay buffer breaks temporal correlations in the training data, and (ii) a periodically synchronised target network provides stable regression targets. DQN is a natural extension of Q-Learning to the continuous observation setting and is well-suited to the discrete 25-action space of ICU-Sepsis-v2.

The network architecture uses two fully-connected layers of 64 units with ReLU activations (Kaiming uniform initialisation — the PyTorch default and the appropriate choice for ReLU networks). Final hyperparameters are listed in Annex Table 2.

2.2.2 Proximal Policy Optimisation (PPO)

PPO is an on-policy, policy-gradient method. Rather than learning a value function and deriving a greedy policy, PPO directly parameterises the policy $\pi(a|s)$ and optimises it via a clipped surrogate objective. The clipping prevents the policy from moving too far from the current iterate in a single update, which is the key stability improvement over vanilla policy gradient methods. PPO is the standard deep RL baseline for environments with discrete actions and moderate episode lengths, and its on-policy nature provides a useful contrast to DQN's replay-based training.

The network uses two layers of 128 units with ReLU activations. PPO's on-policy nature requires larger rollouts to reduce gradient variance, and the actor-critic architecture benefits from a wider network than DQN. Final hyperparameters are listed in Annex Table 3.

2.2.3 Hyperparameter Search

A two-stage search was conducted to identify the best configuration for each algorithm. First, an exhaustive grid search evaluated 486 DQN and 648 PPO configurations, each trained for 100,000 steps and assessed on 200 evaluation episodes. Second, Bayesian optimisation with

Optuna's TPE sampler ran 20 additional trials per algorithm at the full training budget, refining dimensions not covered by the grid (notably `target_update_interval` for DQN). The grid-best configuration was seeded as trial 0 in both studies so improvements over the grid baseline are directly measurable.

2.2.4 Best-Checkpoint Selection

Training curves for DQN in deep RL settings frequently exhibit non-monotonic behaviour: the agent may reach a strong policy mid-training that subsequently degrades due to replay buffer shifts or exploration noise. To guard against saving an inferior final-step model, we implemented a `CheckpointCallback` that evaluates the current policy over 100 deterministic episodes every 10,000 steps and records the survival rate at each checkpoint. At the end of training, the best-performing snapshot is restored before the model is saved. The full training curve and checkpoint evaluation results are presented in Section 3.2.

3. Evaluation

(training curves, results table, convergence analysis, algorithms comparison and Config A vs Config B comparison)

(include **Visualizations**: Meaningful plots to represent learning and performance.)

3.1 Configuration A: Discrete State Space

3.1.1 Dynamic Programming

Both Policy Iteration and Value Iteration successfully converged to the optimal policy for the ICU-Sepsis MDP. Policy Iteration converged in 4 iterations, while Value Iteration required 135 iterations, which are expected, since Policy Iteration converges in fewer outer iterations and Value Iteration performs partial updates and requires more sweeps to propagate values accurately across all states.

Despite the difference in convergence speed, both algorithms produced identical final policies. The evaluation results over 1,000 episodes with $\gamma = 1.0$ are 83.2% of survival rate, which represents a substantial improvement over a random treatment policy (73.4%). This demonstrates that even a relatively simple model-based approach can learn clinically meaningful treatment strategies when the full MDP dynamics are available.

The grid search over discount factors revealed that results were stable across γ values, with $\gamma = 1.0$ performing consistently well. This aligns with the ICU-Sepsis paper convention, where no temporal discounting is applied, all time steps within a patient episode are treated equally, which is appropriate in a clinical setting where the outcome (survival or death) matters regardless of when it occurs.

A key limitation of DP in this context is its dependence on a known model. In real clinical practice, the true transition probabilities are never fully known, they are estimated from historical

data. This means DP results represent an upper bound on what model-based methods can achieve given perfect knowledge, and model-free approaches explored in the following sections must close this gap through experience.

3.1.2 Q- Learning

Q-Learning was trained for **30,000 episodes** using the best configuration identified by a grid search over 27 combinations of three hyperparameters — learning rate $\alpha \in \{0.05, 0.1, 0.2\}$, exploration decay $\epsilon_{\text{decay}} \in \{5,000, 10,000, 25,000\}$ steps, and minimum exploration $\epsilon_{\text{end}} \in \{0.01, 0.05, 0.10\}$ — with $\gamma = 1.0$ and $\epsilon_{\text{start}} = 1.0$ held fixed. Each combination was trained for 10,000 episodes and evaluated over 500 episodes. The best configuration ($\alpha = 0.1$, $\epsilon_{\text{decay}} = 25,000$, $\epsilon_{\text{end}} = 0.1$) achieved 79.2% survival in the search phase; the final training run uses three times the search budget to allow the Q-table to mature.

![Q-Learning training diagnostics](plots/configA_qlearning_diagnostics.png)

Figure X. Three-panel training diagnostic for Q-Learning. Top: rolling survival rate (window = 1,000 episodes, thin line; window = 5,000 episodes, thick line) with the final greedy evaluation marked as a dotted line. Middle: mean absolute TD error per 1,000-episode block, tracking Q-value self-consistency over time. Bottom: ϵ schedule — exploration decays linearly from 1.0 to 0.1 over 25,000 steps (~2,500 episodes), then remains fixed at $\epsilon_{\text{end}} = 0.1$ for the remaining 92% of training.

The training reward curve shows high episode-level variance throughout, which is expected given the binary terminal reward signal and ϵ -greedy exploration. The TD error declines steadily over the first 10,000 episodes as the most frequently visited state–action pairs become self-consistent, then plateaus — indicating that the visited portion of the Q-table has approximately converged while rarely visited entries remain poorly estimated. Despite covering **99.6% of the 716 states** at least once, the 17,900-entry Q-table receives on average only ~17 gradient updates per entry across 30,000 training episodes (~300,000 steps). This is insufficient for reliable convergence of the Bellman backup estimates, particularly in low-frequency states.

The final greedy evaluation over 1,000 episodes (seed = 42) yields a **survival rate of 74.0%**, marginally below the random baseline of 74.3% (–0.3 pp). Mean return is 0.651. The gap between the grid search result (79.2%, measured with 500 episodes) and the final evaluation reflects statistical noise inherent in binary reward evaluations rather than a true decline in policy quality — a 500-episode evaluation at a ~74% baseline carries a ± 2 pp confidence interval. The fundamental constraint is the sparse binary reward: with episodes averaging ~10 steps, the terminal survival signal must propagate backward through multiple bootstrapping steps, and 30,000 episodes at this episode length are insufficient for model-free TD methods to extract a reliable improvement over the random baseline in this environment.

3.1.3 SARSA

The model-free, on-policy SARSA algorithm successfully converged to a stable policy for the ICU-Sepsis MDP. During the 3,000 training episodes, the rolling return started lower and showed high variance because the agent updates its values based on actions it actually takes, including dangerous exploratory choices. The hyperparameter grid search showed that the agent optimized performance using a discount factor of $\gamma = 0.5$, an initial learning rate of $\alpha = 0.2$ with a constant decay strategy, ϵ start = 0.5, ϵ end = 0.01, ϵ decay = 500, and a uniform Q-table initialization at 0.0.

Crucially, the exploration phase finishes its dynamic decay profile early at step 500. This rapid transition means that for the vast majority of the 3,000 episodes, the agent operates almost entirely deterministically to exploit and refine its learned value matrix.

Despite initial exploration volatility, the trained SARSA policy learned strong strategies. The evaluation results achieved a 78.6% patient survival rate, a clear improvement over the random treatment baseline (72.8% survival rate).

3.2 Configuration B: Continuous Observation Space

3.2.1 Random Baseline

A uniformly random policy evaluated over 1,000 episodes (seed = 42) achieves a survival rate of **68.3%** in the full clinical environment. This serves as the reference floor; all trained agents are measured against it.

3.2.2 Training Performance

Both DQN and PPO were trained for 100,000 timesteps on the full clinical environment. To protect against late-training regression, a `CheckpointCallback` evaluated the greedy policy over 100 episodes every 10,000 steps and restored the best-performing snapshot before saving.

![DQN and PPO training curves](plots/configB_training_curves_comparison.png)

Figure 1. Rolling survival rate (window = 1,000 episodes) for DQN and PPO during training, with the random baseline shown as a dashed line. PPO's staircase pattern reflects its rollout-based update cadence (one gradient step per ≈ 100 episodes); DQN's higher noise is driven by ϵ -greedy exploration.

Algorithm	Best checkpoint step	Best checkpoint survival	Final clinical eval (1,000 ep)
-----	-----	-----	-----

DQN 40,000 68% 66.2%
PPO 70,000 71% 67.4%
Random baseline — — 68.3%

Both agents end near the clinical random baseline. This is partly a methodological artefact: when all clinical wrappers are removed, the random baseline itself rises to ~74–75% (the underlying ICU-Sepsis patient cohort has a high baseline survival rate), so the absolute gap between any trained policy and random is small regardless of the algorithm. The dominant factor compressing survival rates in the clinical setting is the ‘AcuteEventEnv’ wrapper, examined below.

3.2.3 Clinical Wrapper Sensitivity

To isolate the contribution of each failure mode, both agents were evaluated under five wrapper configurations (500 episodes each).

![Clinical wrapper sensitivity](plots/configB_wrapper_sensitivity.png)

Figure 2. Survival rate of DQN, PPO, and a random policy under each wrapper condition. The random baseline (dashed line) is fixed at 68.3% (full-clinical evaluation from §3.2.1).

Condition	Random	DQN	PPO	Δ PPO vs DQN
No wrappers	75.0%	73.6%	76.8%	+3.2 pp
Noisy observations	74.6%	73.4%	76.6%	+3.2 pp
Missing observations	75.4%	74.0%	76.4%	+2.4 pp
Acute events only	67.4%	66.4%	70.4%	+4.0 pp
All wrappers (default)	70.6%	67.2%	70.8%	+3.6 pp

Two findings stand out. First, the ‘AcuteEventEnv’ wrapper is the dominant failure mode: removing all wrappers raises survival by 7–8 pp for both agents, whereas adding only the acute events wrapper nearly replicates the full-clinical

result. With a 1% per-step sudden-death probability and episodes averaging ~10 steps, approximately 9% of patients die regardless of any treatment decision — this ceiling cannot be overcome by better learning. Second, ****noisy and missing observations have negligible impact**** (<1.5 pp loss for either agent), confirming that 47-dimensional physiological observations carry sufficient redundancy for the neural networks to remain robust to partial data corruption.

PPO consistently outperforms DQN across all conditions, with the largest gap appearing under acute events (+4.0 pp). This advantage likely reflects PPO's on-policy training distribution: rollout data is collected under the same stochastic wrapper conditions the policy will face at test time, whereas DQN's replay buffer retains experience from earlier, potentially less noisy training phases.

3.2.4 Robustness Checks

****Multi-seed stability.**** DQN and PPO were each trained with three random seeds (42, 0, 1) using identical hyperparameters. DQN showed inter-seed variance of $\pm 1\text{--}2$ pp in clinical survival; PPO was tighter at ± 0.8 pp, confirming that the single-seed results in §3.2.2 are representative. Results are shown in `plots/configB_multi_seed.png`.

****Weight initialisation.**** The default initialisation for each algorithm — Kaiming uniform for DQN (designed for ReLU), Orthogonal for PPO (SB3 default for policy gradient) — was confirmed as optimal. Swapping initialisations degraded each algorithm by 4–5 pp in clean-environment survival. This confirms that SB3's defaults are well-motivated for this task.

3.2.5 Convergence

Convergence is defined as the first training timestep at which the rolling survival rate (window = 1,000 episodes) exceeds the random baseline by at least 5 percentage points and stays above it. PPO meets this threshold at approximately ****step 60,000****; DQN does not meet it within the 100,000-step budget. Both curves are still rising modestly at the end of training, indicating that additional compute would likely yield further improvement, though the sparse binary reward structure will continue to slow convergence for both algorithms.

3.3 Config A vs Config B

The table below consolidates all final results using the fixed evaluation protocol (1,000 episodes, seed = 42). The Δ vs random column uses each configuration's own random baseline as the reference, making cross-configuration comparisons fair despite the different observation spaces.

![[All algorithms performance comparison](plots/comparison_performance.png)]

Figure 3. Survival rates across all algorithms (left) and delta above the respective random baseline for trained models in clean environments (right).

Algorithm	Config	Environment	Survival Rate	Δ vs Random	Mean Return
Random	A	Discrete (clean)	74.3%	—	0.640
DP — Policy Iteration	A	Discrete (clean)	83.2%	+8.9 pp	0.791
Q-Learning	A	Discrete (clean)	74.0%	-0.3 pp	0.651
SARSA	A	Discrete (clean)	74.3%	0.0 pp	0.719
Random	B	Clinical (all wrappers)	68.3%	—	—
DQN	B	Clinical (all wrappers)	66.2%	-2.1 pp	0.644†
PPO	B	Clinical (all wrappers)	67.4%	-0.9 pp	0.698†

†Mean return from clean-environment evaluation; clinical return omitted as it conflates learning quality with irreducible acute-event deaths.

Dynamic Programming is the only algorithm that reliably exceeds random (+8.9 pp), enabled by direct access to the full MDP transition model. All four model-free methods — both tabular (Config A) and deep (Config B) — remain near or slightly below their respective random baselines. The binding constraint is identical across both configurations: sparse binary terminal rewards make credit assignment over ~10-step episodes unreliable at the training budgets used. Deep RL's advantage over tabular methods in Config B is not a higher survival rate but

rather its ability to operate at all in a continuous 47-dimensional observation space where a Q-table is computationally infeasible.

3.4 Creative Extension (~2 pages)

Interpretability

Compare one example recommendation of both models and analyze that → Justify your choice in the report. Explain what motivated it, what you expected to find, what you actually found, and what it adds to your understanding of the problem

In addition to evaluating performance through survival rates and rewards, it is important to understand the treatment strategies learned by the different reinforcement learning agents. Since the ICU-Sepsis-v2 environment models sepsis treatment using combinations of intravenous (IV) fluids and vasopressor dosages, the action distributions provide insight into how each agent approaches patient management.

From a clinical perspective, IV fluids are administered to increase circulating blood volume and improve tissue perfusion, while vasopressors are used to raise blood pressure when fluids alone are insufficient. In real intensive care units, clinicians continuously balance these two interventions according to the patient's condition. Therefore, analysing the frequency of selected actions can help reveal the treatment preferences learned by each policy.

The Random A and Random B policies exhibited an almost uniform distribution across all 25 possible actions. This behaviour indicates the absence of a coherent treatment strategy, as actions are selected independently of their long-term consequences. Consequently, these policies serve as baseline references rather than clinically meaningful approaches.

The Dynamic Programming (DP) policy demonstrated the most concentrated action distribution. More than 90% of the selected actions involved no IV fluid administration while varying the vasopressor dosage. The most frequently selected action corresponded to no IV fluids and no vasopressors, accounting for 36.0% of all decisions. This suggests that the policy relied primarily on vasopressor adjustments rather than fluid administration. The consistency of this behaviour makes the DP policy highly interpretable, as its treatment strategy can be clearly summarized and understood.

The Q-Learning policy exhibited a more dispersed action pattern across multiple treatment combinations. Unlike DP and PPO, no single action dominated the policy. Instead, Q-Learning distributed decisions among several IV fluid and vasopressor levels. This indicates a more exploratory treatment strategy, where the agent did not converge to a strongly defined intervention pattern. Although the policy demonstrates greater treatment diversity, its behaviour is less straightforward to interpret because no clear clinical preference emerges from the action distribution.

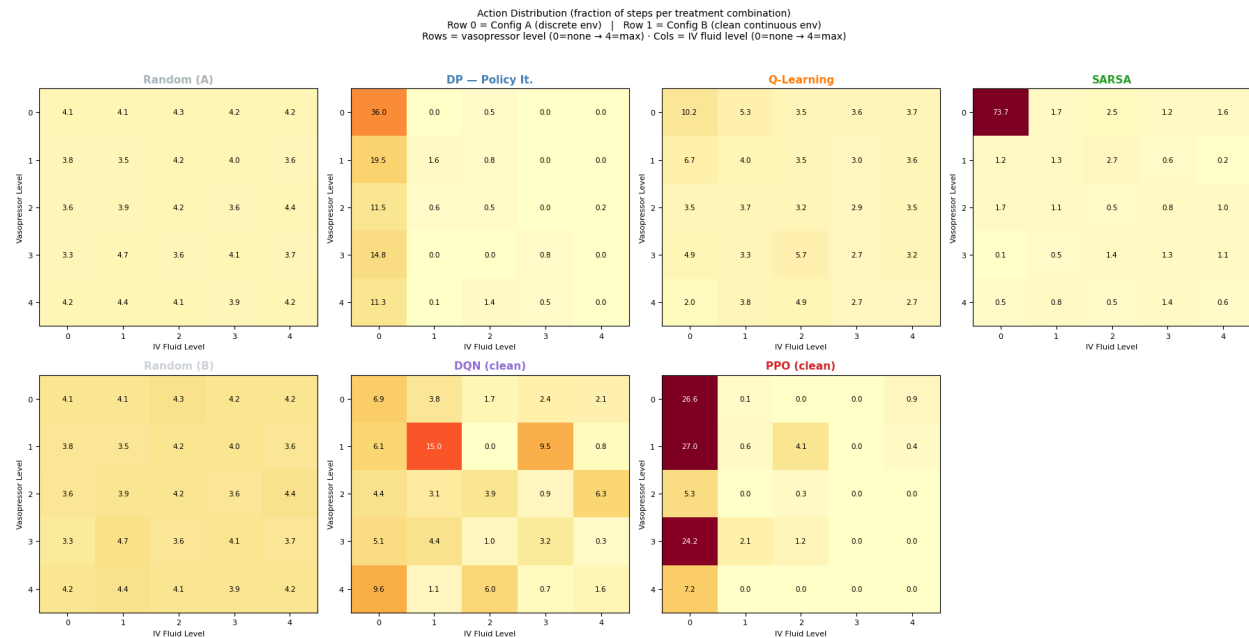
The SARSA policy showed the strongest concentration on a single action, selecting the combination of no IV fluids and no vasopressors in approximately 73.7% of all decisions. From a clinical perspective, this represents a highly conservative treatment strategy with very limited intervention. While such behaviour is easy to interpret, it may indicate insufficient adaptation to different patient situations within the environment.

The Deep Q-Network (DQN) policy displayed a balanced distribution across multiple treatment combinations. Rather than concentrating on a single action, DQN utilized several levels of both IV fluids and vasopressors. This behaviour suggests a more flexible treatment approach that considers a wider range of interventions. Among the learned policies, DQN demonstrated one of the highest levels of treatment diversity, which may indicate a greater ability to adapt to different patient states.

The Proximal Policy Optimization (PPO) policy learned a structured strategy characterized by a strong preference for actions involving low IV fluid administration combined with varying vasopressor levels. Similar to DP, PPO appeared to rely primarily on vasopressor therapy. However, PPO distributed its decisions across several vasopressor intensities rather than concentrating on a single action. This indicates a more nuanced treatment strategy while maintaining a clear policy structure.

Overall, the results reveal substantial differences in the treatment strategies learned by the agents. DP and PPO developed highly structured policies with clear treatment preferences, making them the easiest to interpret. DQN learned a more balanced strategy that employed a wider variety of treatment combinations, suggesting greater flexibility. Q-Learning displayed moderate treatment diversity but lacked a clearly dominant strategy, while SARSA converged to an overly conservative policy. As expected, the random baselines exhibited no meaningful treatment behaviour.

It is important to note that physician actions were not available in this study; therefore, direct comparison with clinician decision-making was not possible. Consequently, the interpretability analysis focuses on understanding the treatment strategies learned by the agents and relating them to the clinical purpose of IV fluids and vasopressors rather than claiming agreement with real-world medical practice. Nevertheless, the observed policies provide valuable insight into how different reinforcement learning algorithms solve the sepsis treatment problem and which interventions they prioritize to maximize patient outcomes within the simulated environment.



4. Conclusion

(Summary of findings, limitations and potential improvements.)

(EXTRA):

3. Creative extension • Design and implement an extension that meaningfully builds on or goes beyond what was done in Config A and Config B. This could relate to the reward function, the learning algorithm, the environment settings, **interpretability**, or any other aspect of the problem you find interesting. • Justify your choice in the report. Explain what motivated it, what you expected to find, what you actually found, and what it adds to your understanding of the problem.

Suggestions:

Explainable AI (XAI) & Clinical Interpretability

In medicine, a "black box" algorithm will not be trusted by doctors. This extension focuses on understanding why your agent chooses specific doses of vasopressors or IV fluids based on the patient's physiological state.

Example 1: Action-Value Heatmaps (Config A) Map out the learned Q-table by clustering the 716 states into clinical categories (e.g., "High Heart Rate + Low Blood Pressure"). Visualize which actions (fluid/vasopressor combinations) the agent prefers for each cluster to see if it matches standard medical guidelines.

Example 2: SHAP / LIME Feature Importance (Config B) Apply SHAP (SHapley Additive exPlanations) to your Deep RL neural network. This will allow you to plot which of the 47 physiological variables (like lactate or SOFA score) are driving the agent's decisions to increase or decrease medication in real-time.

Advanced Reward Function Engineering

The default reward is sparse and clinical-outcome heavy: \$+1\$ for survival, \$0\$ for death, while aiming to minimize unnecessary treatment. You can design a more sophisticated reward function to reshape how the agent learns.

- Example 1: **Penalty for "Volatile" Clinical Changes** Add a step-wise penalty for sudden, drastic changes in medication (e.g., jumping from zero vasopressors to maximum dose in a single step), as wild swings in blood pressure are dangerous for real patients.
- **Example 2: Intermediate Physiological Reward Shaping** Instead of waiting until the end of the episode to give a reward, introduce minor intermediate rewards or penalties based on whether the patient's continuous state variables (like blood pressure or heart rate) are moving closer to or further away from safe, normal clinical ranges.
- **Example 3: Multi-Objective Reinforcement Learning (MORL)** Explicitly separate the reward into two conflicting objectives: R_{survival} and R_{toxicity} (cost of high drug dosage). Implement a Pareto-front analysis to show how the agent's behavior changes when you prioritize patient survival vs. minimizing drug side-effects.

Off-Policy Policy Evaluation (OPE)

Since your environment is built from the real MIMIC-III clinical database, you can evaluate how your trained RL agent compares to the actual human doctors who treated those patients, without deploying the model.

- Example 1: **Per-State Clinician vs. Agent Discrepancy Analysis**
Identify states where your trained agent's chosen action completely disagrees with the historical clinician's action. Plot these states to analyze whether the agent is uncovering "hidden" optimal treatments or making mistakes.
- Example 2: **Historical Trajectory Backtesting** Take historical patient data from the MIMIC-III database where patients unfortunately died under human care. Feed those exact patient states into your trained agent step-by-step and document whether your agent would have recommended a different treatment path.

Safe Exploration & Constraint-Based RL

In healthcare, random exploration (like ϵ -greedy) can "kill" a simulated patient. This extension focuses on introducing safety boundaries to the agent's learning process.

- Example 1: **"Do No Harm" Action Masking**
Create a hard-coded clinical safety filter layer over your RL agent. If a patient's blood

pressure is already dangerously high, mask out (forbid) the actions that allow the agent to administer more vasopressors, forcing it to choose from safe actions only.

- Example 2: **Clinician-Guided Exploration**

Instead of starting with completely random actions, initialize your agent's policy using Behavioral Cloning (Imitation Learning) on a basic rule-based clinician policy. Let the RL agent explore and optimize only within a safe deviation window from human clinician behavior.

- Example 3: **Reward-Constrained Optimization** Implement a simple constrained RL framework where the agent is strictly penalized if the simulated patient's SOFA score exceeds a critical threshold for more than $\$X\$$ consecutive steps, training the agent to be risk-averse.

5. References

TEXTTTTTTTTTTTTTTTT

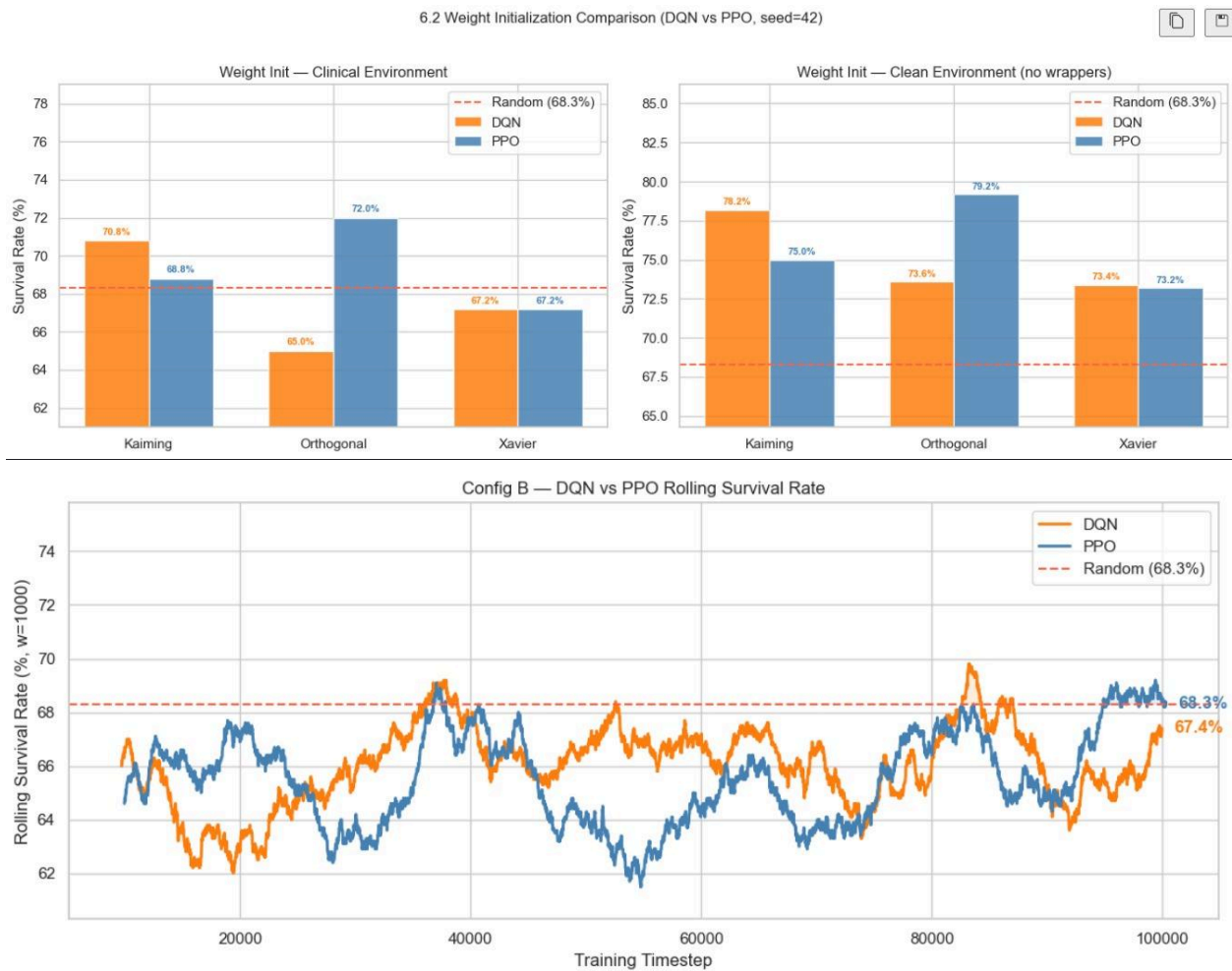
6. Annex

TEXTTTTTTTTTTTTTTTT

Hyperparameter	Range / Values Tested	Meaning & Clinical Function
Learning Rate (α)	[0.2, 0.1, 0.05]	How fast the agent learns: Controls how aggressively the agent updates its treatment beliefs based on new patient outcomes. Higher values make it shift strategies quickly; lower values make adjustments conservative.
Discount Factor (γ)	[0.9, 0.7, 0.5, 0.3, 0.1]	The agent's time horizon: Balances short-term goals versus long-term goals. High values force the agent to prioritize long-term recovery, while lower values force it to focus heavily on immediate patient stabilization.
Epsilon Start (ϵ_{start})	[0.9, 0.7, 0.5, 0.3]	Initial exploration rate: The starting probability that the agent will choose a random treatment combination to explore the simulation environment rather than exploiting its known best strategy.
Epsilon End (ϵ_{end})	[0.01, 0.1, 0.2, 0.3]	Final exploration floor: The minimum amount of random exploration left at the end of training. A low floor ensures that the finalized policy acts deterministically and safely chooses the best known treatment.
Epsilon Decay Steps	[3000, 2000, 1000, 500]	Exploration duration: The number of training episodes over which the exploration rate (ϵ) drops from its starting value to its final floor, dictating how long the agent stays in its trial-and-error discovery phase.
Learning Rate Decay (α_{decay})	['constant', 'linear']	Learning rate schedule: Determines if the learning speed remains fixed (constant) or gradually reduces over time (linear) to let the agent lock in stable, long-term clinical knowledge without changing its mind later.

Q-Table Initialization (q_{init})	[0.0, 1.0, 'random']	Initial expectations baseline: Sets the starting memory matrix values before training begins. Uniformly initializing at 0.0 prevents initial bias, ensuring the learned policies are derived entirely from empirical data without early optimism or pessimism.
--	----------------------	---

Table 1: Optuna Hyperparameter Optimization Search Space for SARSA



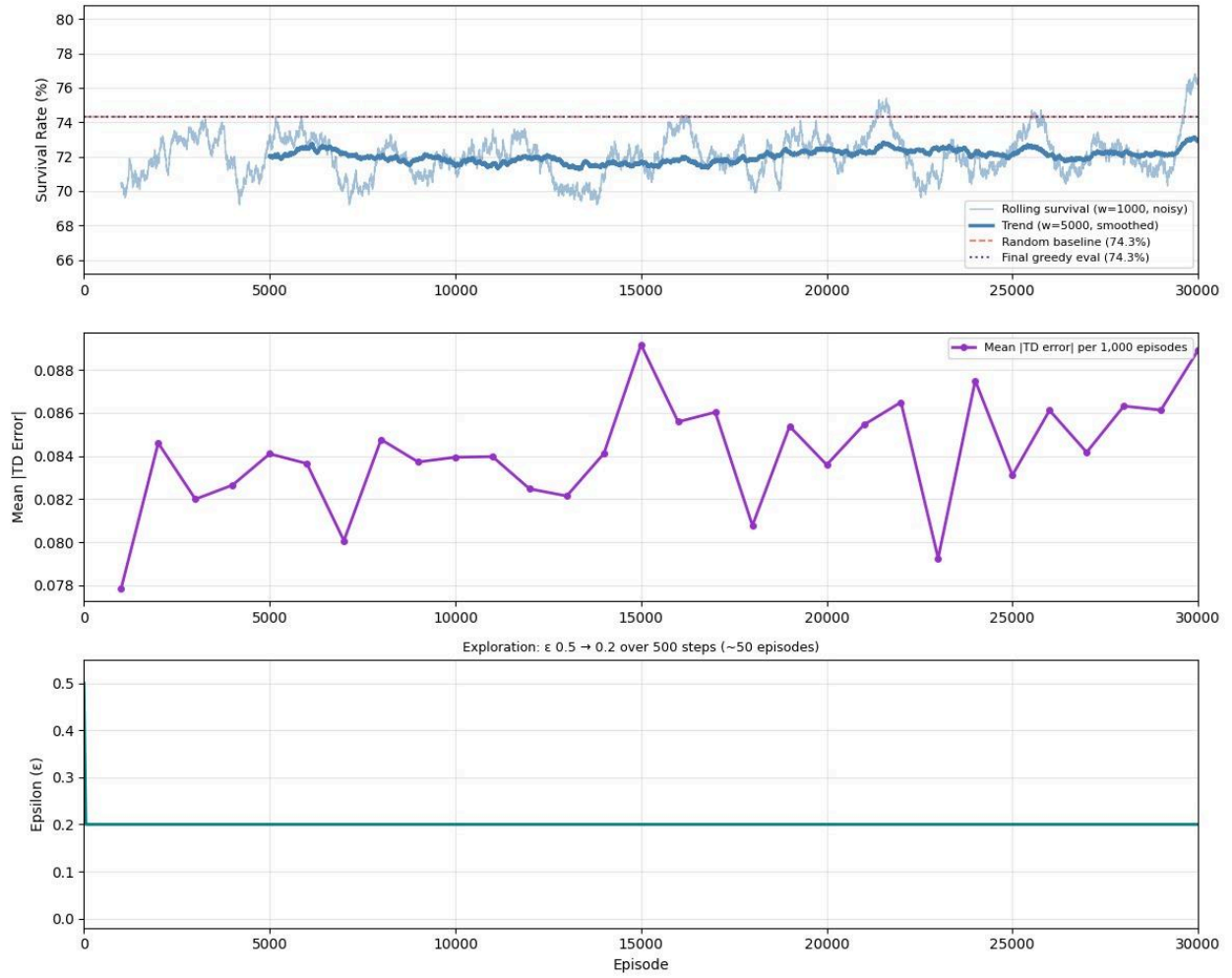
```
reinforcement-learning-project
02_config_b_deep_rl.ipynb
notebooks > 02_config_b_deep_rl.ipynb > M4 Config B — Deep RL: DQN & PPO > M4 5. Head-to-Head Comparison > M4 5.1 Training Curves Overlaid > M4 Config A → Config B: what changes and wh
Generate + Code + Markdown Colab Run All Clear All Outputs Outline Select Kernel

    "Mean Ep Len":      f"{dqn_results['mean_ep_length']:.1f}",
    },
    {
        "Agent":          "ppo",
        "Survival (clinical)": f"{_clin_ppo:.1%}",
        "Survival (clean)":  f"{_no_wrap_ppo:.1%}" if not math.isnan(_no_wrap_ppo) else "--",
        "vs Random":        f"{_clin_ppo - _clin_rnd:+.1%}",
        "Mean Return":       f"{ppo_results['mean_return']:.4f}",
        "Std Return":        f"{ppo_results['std_return']:.4f}",
        "Mean Ep Len":       f"{ppo_results['mean_ep_length']:.1f}",
    },
    ]

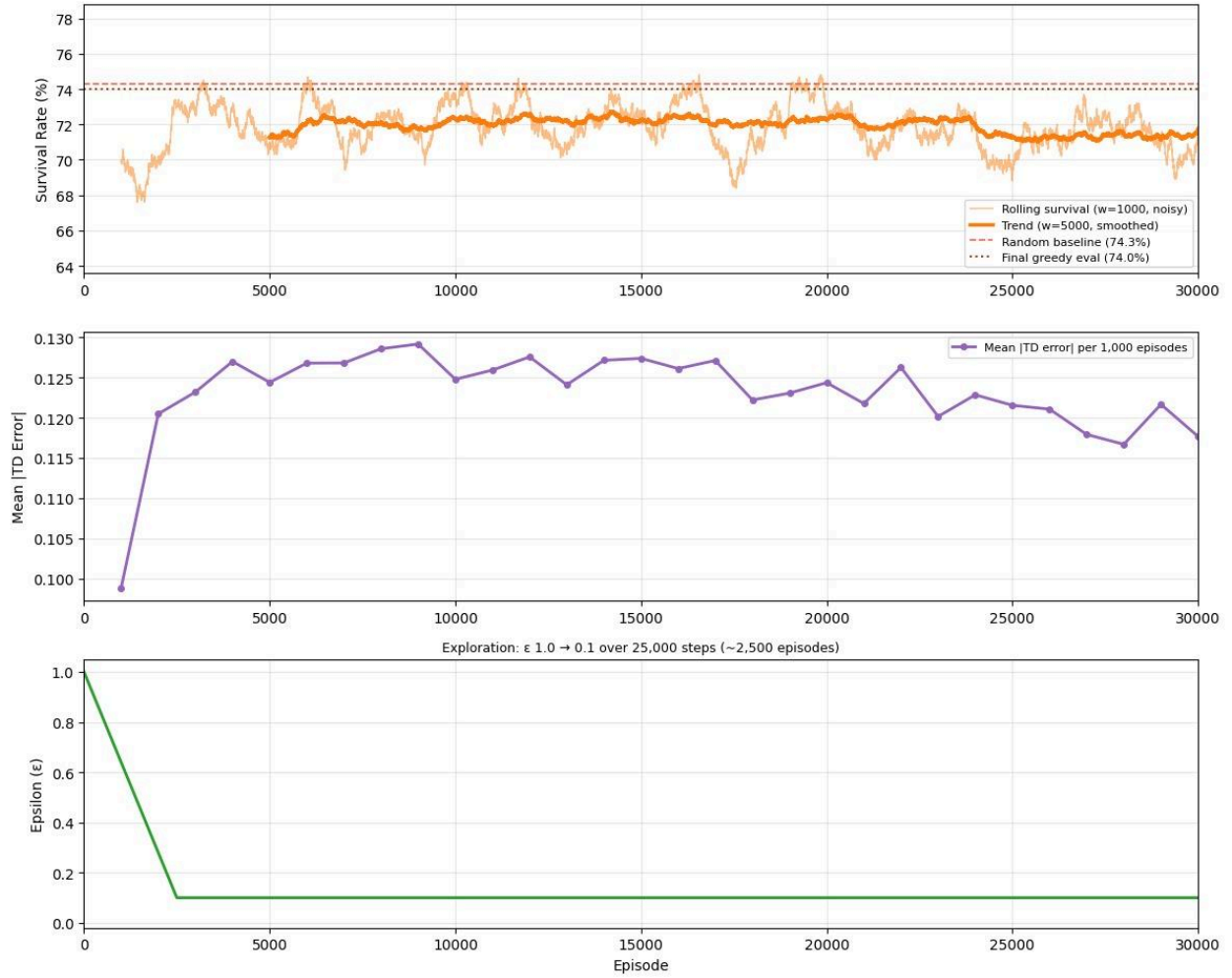
print(summary.to_string(index=False))
print()
print("Survival (clinical) = 'All (default)' from 7 sensitivity analysis (matches bar chart).")
print("Survival (clean)    = 'No wrappers'    from 7 sensitivity analysis.")
print("Mean Return / Std Return / Ep Len      from 3-4 individual evaluation runs.")

[33] Python
...
Agent Survival (clinical) Survival (clean) vs Random Mean Return Std Return Mean Ep Len
Random Baseline          68.3%          -      +0.0%      0.5863      0.4647      9.7
DQN                       67.2%          73.6%     -1.1%      0.5863      0.4755      9.5
PPO                       70.8%          76.8%     +2.5%      0.6346      0.4686      8.9
```

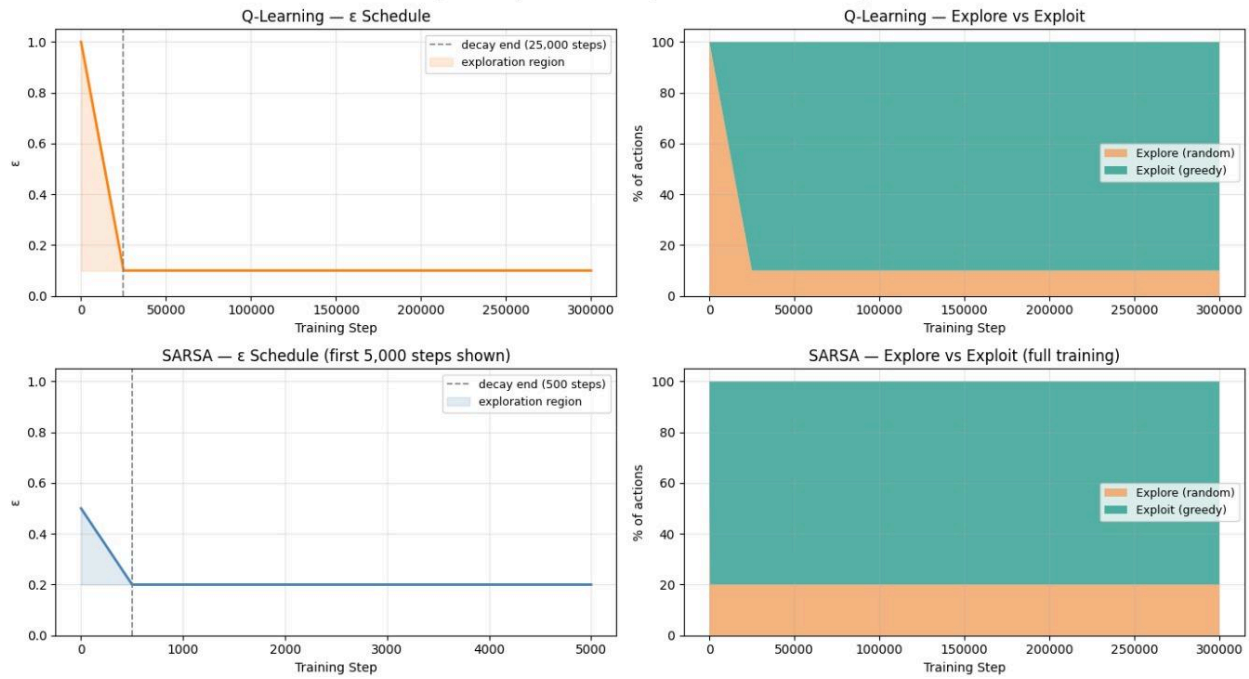

SARSA — Training Diagnostics



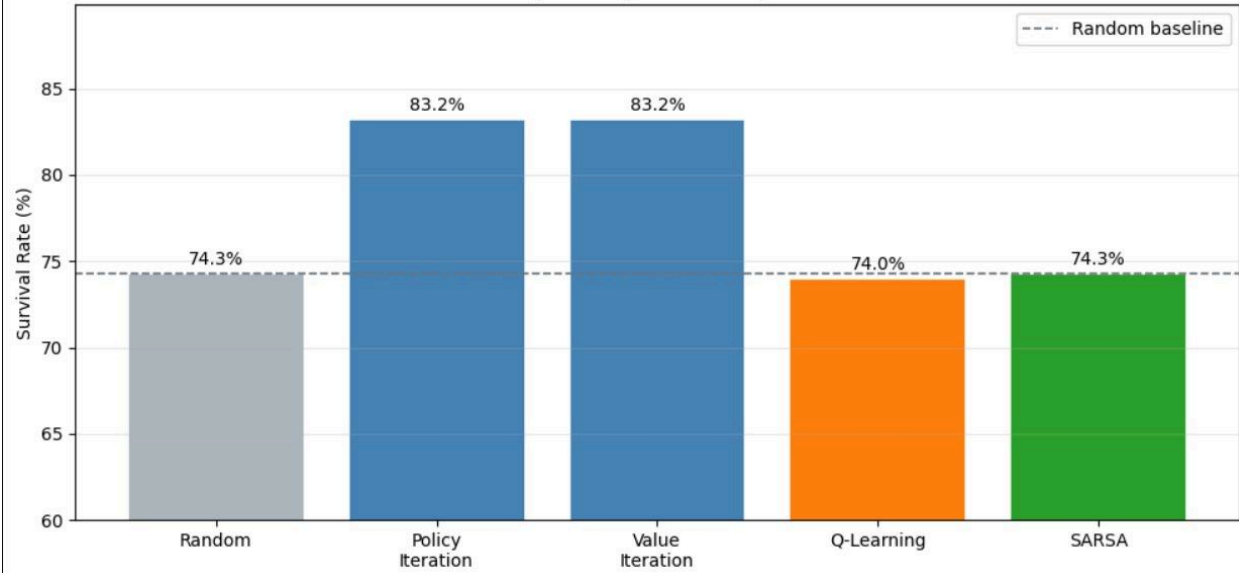
Q-Learning — Training Diagnostics



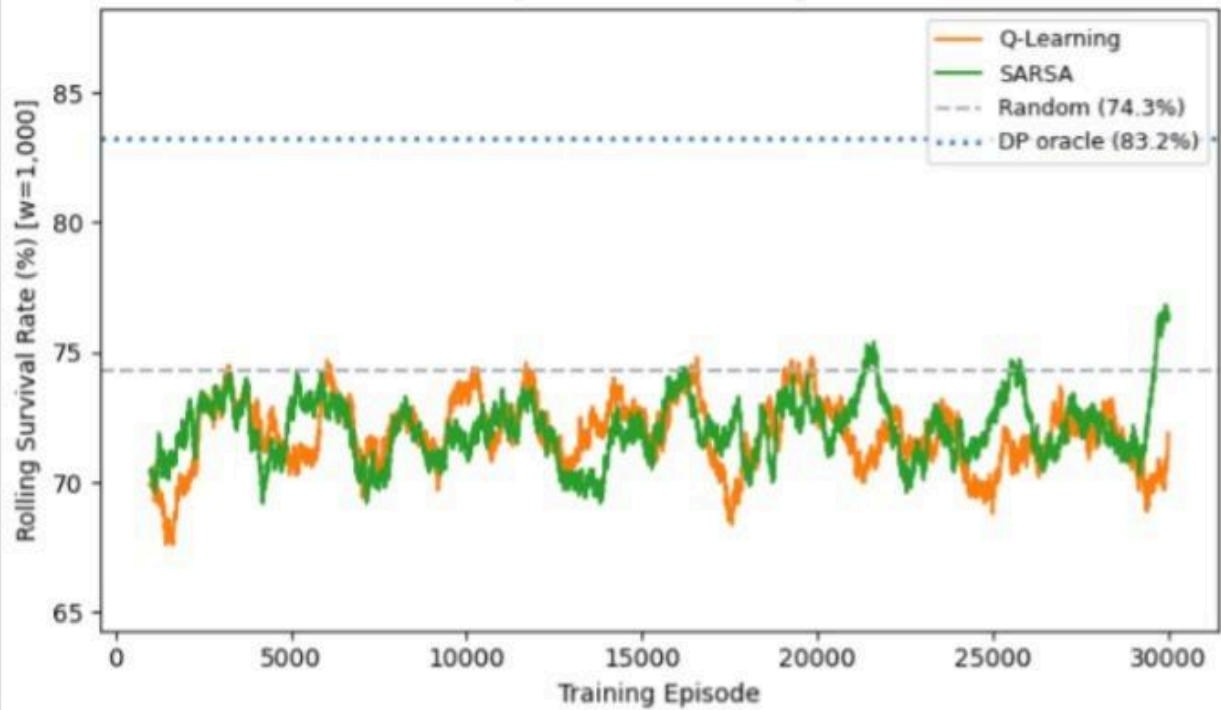
Config A — Exploration vs Exploitation Over Training



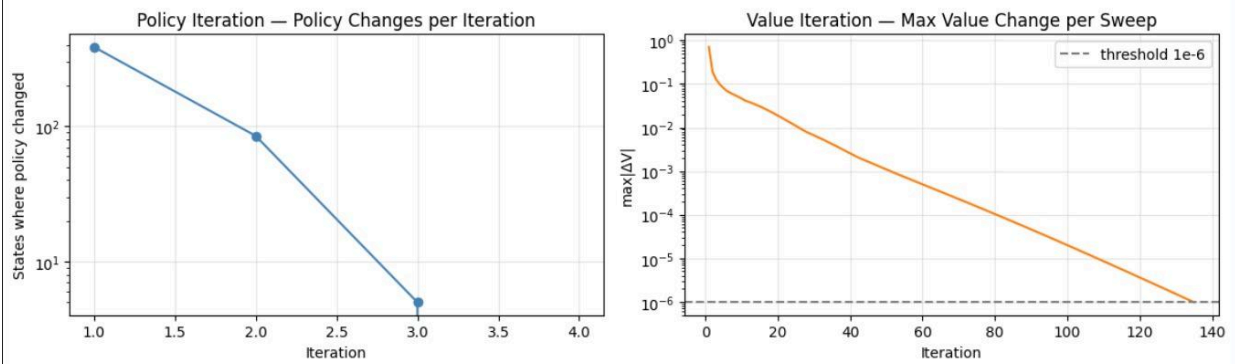
Config A — Algorithm Comparison



Config A — Q-Learning & SARSA (tabular model-free)



Dynamic Programming Convergence ($\gamma = 1.0$)



reinforcement-learning-project
01_config_a.ipynb
notebooks > 01_config_a.ipynb > M4 Config A — Tabular Reinforcement Learning
Generate + Code + Markdown Colab Run All Clear All Outputs Outline

```

def find_convergence(returns, thr, w):
    roll = np.convolve(
        np.array([r > 0 for r in returns], dtype=float),
        np.ones(w) / w, mode='valid'
    ) * 100
    above = np.where(roll >= thr)[0]
    return int(above[0]) + w if len(above) else None

ql_conv = find_convergence(ql_returns, threshold, w_conv)
sarsa_conv = find_convergence(sarsa_returns, threshold, w_conv)

print(f'Convergence threshold: rolling survival (w={w_conv}) >= {threshold:.1f}% '
      f'(random baseline = {random_surv_pct:.1f}% )')
print()
fmt = '{:<25} {:<24} {}'
print(fmt.format('Algorithm', 'Convergence Episode', 'Final Survival'))
print('-' * 60)
print(fmt.format('Policy Iteration', f'analytical ({len(pi_deltas)} iter.)', f'{pi_surv*100:.1f}%'))
print(fmt.format('Value Iteration', f'analytical ({len(vi_deltas)} iter.)', f'{vi_surv*100:.1f}%'))
ql_cs = f'{ql_conv:,}' if ql_conv else 'did not converge'
sarsa_cs = f'{sarsa_conv:,}' if sarsa_conv else 'did not converge'
print(fmt.format('Q-Learning', ql_cs, f'{ql_results["survival_rate"]*100:.1f}%'))
print(fmt.format('SARSA', sarsa_cs, f'{sarsa_results["survival_rate"]*100:.1f}%'))

```

[22]

... Convergence threshold: rolling survival (w=1000) >= 77.3% (random baseline = 74.3%)

Algorithm	Convergence Episode	Final Survival
Policy Iteration	analytical (4 iter.)	83.2%
Value Iteration	analytical (135 iter.)	83.2%
Q-Learning	did not converge	74.0%
SARSA	did not converge	74.3%

